Written Work #0

Linear Algebra Refresher

CSC/DSE 489/589 — Scientific Computing — Fall 2026

Assigned: Mon Aug 24 **Due:** Wed Sep 9, start of class

Pen and paper. Work these by hand. You are welcome to check yourself in Julia, but the hand computation is the point and is what is graded.

Notation. Matrices are bold uppercase ($\mathbf{A}$, $\mathbf{U}$), vectors are bold lowercase ($\mathbf{x}$, $\mathbf{b}$), and scalars are plain italic (x_1, λ, m_{ij}). This course uses that convention throughout, so it is worth getting used to now.

Purpose

MATH 315 is a prerequisite, so most of what follows is review. However, you will need linear algebra "on your fingertips" for this course, and you might not have practiced it recently. This assignment is to help shake the rust off, with an emphasis on concepts important to the course. Everything below gets used, most of it within three weeks.

Chip away at this alongside the Julia material. **Do not save it for the night before**.

Sections marked ▲ cover material that MATH 315 sometimes runs out of time for. Those sections open with a refresher, and a worked example where one helps. If a ▲ section is genuinely unfamiliar, that is expected, and it is much better to discover it now — come to office hours.

A few problems deliberately stop short of an answer, and say so where they do. Those are not oversights and they are not trick questions: they are the places where this course starts, and you will get more out of the lecture for having sat with the question first. Write down what you think and move on.

Submission

Hand in on paper, **stapled**, at the start of class on Wednesday, September 9. Show your work: a correct final answer with no derivation earns little credit, and a wrong answer with sound reasoning earns most of it.

If you are stuck for more than twenty minutes on any single problem, come to office hours or ask questions in class. Being stuck now is cheap. Being stuck in October is not.

1 Matrix multiplication, four ways

You know how to multiply matrices. What this course needs is the ability to see the *same product* four different ways, because different algorithms exploit different views.

For $\mathbf{A}$ $(m \times k)$ and $\mathbf{B}$ $(k \times n)$, the product $\mathbf{AB}$ is simultaneously:

(1) **Dot products.** Entry (i, j) is row i of $\mathbf{A}$ dotted with column j of $\mathbf{B}$.

(2) **Columns.** Column j of $\mathbf{AB}$ is $\mathbf{A}$ times column j of $\mathbf{B}$: a linear combination of the columns of $\mathbf{A}$.

(3) **Rows.** Row i of $\mathbf{AB}$ is row i of $\mathbf{A}$ times $\mathbf{B}$: a linear combination of the rows of $\mathbf{B}$.

(4) **Sum of outer products.** $\mathbf{AB} = \sum_{p=1}^{k} (\text{col } p \text{ of } \mathbf{A})(\text{row } p \text{ of } \mathbf{B})$.

1.1 For

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix},$$

compute $\mathbf{AB}$ all four ways. Show the intermediate quantities in each. For (4), write out both outer products as full 2×2 matrices before summing.

1.2 Matrix-vector multiplication, $\mathbf{Ax}$, can be viewed as a *linear combination of the columns of* $\mathbf{A}$. Write that combination explicitly for the $\mathbf{A}$ above and $\mathbf{x} = \begin{bmatrix} 2 & -1 \end{bmatrix}^{\mathsf{T}}$.

1.3 Suppose $\mathbf{A}$ is 1000×1000 and $\mathbf{x}$ is a vector. Views (1) and (2) suggest different orders in which to loop over the entries of $\mathbf{A}$ when forming $\mathbf{Ax}$. Write both loop orderings in pseudocode.

> Keep your answer to the previous problem. We return to it in Unit 1, when we reach the memory hierarchy — one of the two orderings turns out to be roughly ten times faster than the other, and the reason is not arithmetic.

2 Rank and linear independence

Short section, but the vocabulary here is what lets us say precisely when a system has a solution, when that solution is unique, and — later in the course — what goes wrong when a matrix is *nearly* rank deficient.

Refresher

Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are **linearly independent** if the only scalars $c_1, \ldots, c_k$ with $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ are all zero. Otherwise one of them is a combination of the others and is, in a precise sense, redundant.

The **column space** of $\mathbf{A}$ is the span of its columns — equivalently, by view (2) of §1, the set of all vectors $\mathbf{Ax}$. So $\mathbf{Ax} = \mathbf{b}$ has a solution exactly when $\mathbf{b}$ lies in the column space of $\mathbf{A}$.

The **rank** of $\mathbf{A}$ is the number of linearly independent columns, which is also the dimension of the column space (and, as it happens, the number of independent rows). A matrix with $\text{rank}(\mathbf{A}) = n$ for its n columns has **full column rank**.

Problems

2.1 Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix}.$$

Determine $\text{rank}(\mathbf{A})$. Then exhibit the redundancy explicitly: write the third column as a linear combination of the first two.

2.2 Using the same $\mathbf{A}$, decide for each of

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}, \qquad \mathbf{b}_2 = \begin{bmatrix} 1 \\ 3 \\ 0 \end{bmatrix}$$

whether $\mathbf{Ax} = \mathbf{b}$ has a solution, and say why in terms of the column space. For whichever is solvable, find one solution.

2.3 In problem 2.2, is the solution you found unique? Justify your answer using $\text{rank}(\mathbf{A})$ rather than by solving anything further.

2.4 Now let $\mathbf{R} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, which has rank 1. Change the bottom-right entry from 4 to $4 + 10^{-10}$. What is the rank now?

> That last problem is not a trick. By the definition you learned in MATH 315, an arbitrarily small change to one entry took the rank from 1 to 2 — so rank, as a count of pivots, is useless for a matrix whose entries came from measurement or from floating-point arithmetic. Deciding what "rank" should mean in that setting is what takes us to the singular value decomposition, later in Unit 1.

3 Triangular systems and counting work

Two solves by hand, then the habit this course cares about most: knowing what an algorithm costs before you run it.

> **Counting convention.** A **flop** is one floating-point operation. An addition, a subtraction, a multiplication, and a division each count as one flop.
>
> (Note that this is a hardware-agnostic approximation of arithmetic work. For example, modern processors can fuse multiply-add, $ax+b$, into a single instruction and division is more expensive. We use this simple counting convention for convenience.)

3.1 Solve by hand, using forward substitution:

$$\begin{bmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ -1 & 2 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ 3 \end{bmatrix}.$$

3.2 Solve by hand, using back substitution:

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 3 & -1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 2 \end{bmatrix}.$$

3.3 Count the flops required to solve a general $n \times n$ lower-triangular system by forward substitution. Break the count down by operation — multiplications, subtractions, divisions — then total them. Give an exact count as a polynomial in n, not a summation. Hint: $1+2+\cdots+n = n(n+1)/2$.

Then suppose the matrix, call it $\mathbf{L}$, is *unit* lower triangular — every diagonal entry equal to 1. What happens to your count, and why? You will meet this matrix again in Section 4.

3.4 How many flops does it take to multiply an $n \times n$ matrix by a vector? Again, give the exact count as a polynomial.

3.5 How many flops does it take to multiply two $n \times n$ matrices? Exactly, again — the figure you will see quoted for this one is rounded off, and yours should not be.

4 Gaussian elimination is a factorization

This is the bridge between MATH 315 and the way Unit 1 treats elimination, and it is the most important section in this assignment.

You learned Gaussian elimination as a procedure for solving systems. This course treats it as a procedure for *factoring a matrix*, which is a different emphasis: you factor once, then reuse the factors many times.

4.1 Apply Gaussian elimination (no row swaps are needed) to

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix}.$$

Record every multiplier as you go. Write each row operation in the form

$$R_i - m_{ij} R_j \;\rightarrow\; R_i,$$

and call m_{ij} the multiplier that eliminates the entry in row i, column j. For example, clearing the 4 in position $(2,1)$ takes $R_2 - 2R_1 \rightarrow R_2$, so $m_{21} = 2$.

Mind the sign: m_{ij} is the factor you *subtract*, not the coefficient you add.

4.2 Write down the upper triangular result $\mathbf{U}$.

4.3 Build the matrix $\mathbf{L}$ with 1's on the diagonal and m_{ij} in position (i, j) below it. Multiply $\mathbf{LU}$ by hand and confirm you recover $\mathbf{A}$.

4.4 In your own words, one short paragraph: you have now solved *no* linear system, yet you have done all of the elimination work. What have you actually gained, and when would that be worth it?

5 Inverses, and the habit of not using them

5.1 Consider the linear system $\mathbf{Ax} = \mathbf{b}$ with

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 4 \\ -2 \\ 7 \end{bmatrix}.$$

Solve $\mathbf{Ax} = \mathbf{b}$ two ways: (a) form $\mathbf{A}^{-1}$ then multiply by $\mathbf{b}$, and (b) Gaussian elimination and backward-substitution. Confirm they agree.

5.2 Which route was more work? One short paragraph, and keep it informal — no operation counts. Say what you actually spent your time on, and what you would expect to happen to each route if $\mathbf{A}$ were 100×100 instead of 3×3. We put exact numbers to this in Unit 1.

6 ▲ Symmetry, definiteness, and quadratic forms

Refresher

A symmetric matrix $\mathbf{A}$ (meaning $\mathbf{A} = \mathbf{A}^\mathsf{T}$) is **positive definite** if $\mathbf{x}^\mathsf{T}\mathbf{Ax} > 0$ for every nonzero vector $\mathbf{x}$. Two equivalent tests:

- $\mathbf{x}^\mathsf{T}\mathbf{Ax} > 0$ for all $\mathbf{x} \neq \mathbf{0}$ (the definition);
- every eigenvalue of $\mathbf{A}$ is positive.

They are equivalent, and for a 2×2 matrix you can check both in under a minute — which is what the problems below ask you to do. Their purposes differ, though. The *definition* is what proofs use. The *eigenvalue* test is the one that keeps mattering: the ratio of the largest eigenvalue to the smallest turns out to govern how fast several of this course's algorithms converge.

A **quadratic form** is a function $q(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{Ax}$. For symmetric 2×2 $\mathbf{A} = \left[\begin{smallmatrix} a & b \\ b & c \end{smallmatrix}\right]$, this expands to $ax_1^2 + 2b\,x_1x_2 + cx_2^2$.

Worked example

Let $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$. Then

$$\mathbf{x}^\mathsf{T}\mathbf{Ax} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2x_1 \\ 5x_2 \end{bmatrix} = 2x_1^2 + 5x_2^2,$$

which is positive whenever $\mathbf{x} \neq \mathbf{0}$. The eigenvalues are 2 and 5, both positive. Both tests agree.

Problems

6.1 Let $\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$. Write out $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ explicitly as a polynomial in x_1, x_2.

6.2 Show $\mathbf{A}$ is positive definite by both tests above. For the first, complete the square.

6.3 Show that $\mathbf{B} = \begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix}$ is *not* positive definite by exhibiting a specific vector $\mathbf{x}$ with $\mathbf{x}^\mathsf{T}\mathbf{B}\mathbf{x} < 0$.

> We *love* symmetric matrices, and we really love positive definite ones. A real symmetric matrix always has real eigenvalues and admits an orthonormal basis of eigenvectors — which is not something a nonsymmetric matrix can be counted on to have. Symmetry also buys an LU-like factorization at half the work, which we build in Unit 1. And positive definiteness buys one thing more: that factorization needs no row swaps at all, which is why it is the one we reach for whenever we are allowed to.

7 ▲ Block matrices

Rarely emphasized in MATH 315, and central to how fast linear algebra is actually implemented.

Refresher

Matrices may be partitioned into submatrices and multiplied blockwise, *provided the partitions are compatible*. If

$$\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}, \qquad \mathbf{N} = \begin{bmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{bmatrix},$$

with block dimensions that line up, then

$$\mathbf{MN} = \begin{bmatrix} \mathbf{AE} + \mathbf{BG} & \mathbf{AF} + \mathbf{BH} \\ \mathbf{CE} + \mathbf{DG} & \mathbf{CF} + \mathbf{DH} \end{bmatrix}$$

— the same formula as 2×2 scalar multiplication, with blocks in place of numbers and the order of every product preserved. Blocks do not commute.

Problems

7.1 Let

$$\mathbf{M} = \left[\begin{array}{cc|cc} 1 & 2 & 0 & 0 \\ 3 & 4 & 0 & 0 \\ \hline 0 & 0 & 5 & 6 \\ 0 & 0 & 7 & 8 \end{array}\right], \qquad \mathbf{N} = \left[\begin{array}{cc|cc} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ \hline 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{array}\right].$$

Compute **MN** blockwise using the formula above, then verify by multiplying the full 4×4 matrices directly.

7.2 In problem 7.1, several blocks were zero. How much arithmetic did the block view let you skip?

7.3 Suppose you have a fast routine for multiplying 64×64 matrices and you need to multiply two 512×512 matrices. Describe, in two or three sentences, how you would use the block view to do it. This is essentially what every optimized linear algebra library does; we reach the reason in Unit 1, with BLAS and blocked algorithms.

8 How much can a matrix stretch a vector?

8.1 Let $\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$. For each of the following unit vectors $\mathbf{x}$, compute the ratio $\|\mathbf{A}\mathbf{x}\|_2 / \|\mathbf{x}\|_2$:

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}, \quad \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}.$$

8.2 Over *all* nonzero $\mathbf{x}$, what is the largest this ratio can be? The smallest? Justify your answer — a picture is fine.

8.3 Compute the ratio of your two answers to problem 8.2.

> That ratio has a name, and it is the most important single quantity in the first half of this course — it governs how much a linear system can amplify error, which algorithms are trustworthy, and how fast several methods converge. We meet it almost immediately after this assignment is due.